

Technical summary and feature proposal

Part 1: Technical Architecture & Data Flow

The application operates in two distinct phases: **the Offline Setup, which constructs the core model, and the Online Runtime, which enables** interactive chat functionality.

1. The Tech Stack.

- **Frontend UI:** Streamlit (Python-based reactive web framework).
- **Document Processor:** PyMuPDF (extracts raw text) & LangChain (splits text using `RecursiveCharacterTextSplitter`).
- **Vector Database:** ChromaDB (local persistence).
- **Embedding Model:** `sentence-transformers` (`all-MiniLM-L6-v2` via Hugging Face) converts text into numerical vectors.
- **Emotion Engine:** Hugging Face `transformers` pipeline using a fine-tuned `distilroberta-base` model.
- **LLM Engine:** Groq API powering Meta's `Llama-3.3-70b-versatile` for fast, intelligent generation.

2. The Data Flow (Step-by-Step)

For example, when a student inputs: *"I keep getting syntax errors with lists, I feel so stupid,"* the following processes occur within milliseconds:

1. Dual-Track Processing:

- **Track A (Emotional):** The raw text is processed by the local Roberta model, which analyzes sentiment and assigns the label `"High Anxiety (sadness)"`.
- **Track B (Logical):** The raw text is sent to Groq with explicit instructions to remove emotional content and extract *only* the relevant technical keyword. The output is `"lists syntax"`.

2. Vector Retrieval (RAG):

- The clean keyword (`"lists syntax"`) is converted into a vector embedding.
- ChromaDB searches its database of textbook chunks to find the closest matches and returns the corresponding textbook paragraphs.

3. Prompt Synthesis:

- The Streamlit application functions as the orchestrator, aggregating the following components:
- The system instructions (Your strict tutor rules).
- The chat history (Memory).
- The user's actual prompt.
- The detected emotion.
- The retrieved textbook context.

4. **Generation:** Groq processes the contextualized prompt and generates a response designed to address the user's emotional state prior to delivering a factual, textbook-based explanation.

5. **State Management:** The Streamlit application appends the final interaction to `st.session_state.chat_history` and updates the web user interface accordingly.

Part 2: Designing for Human-Centered AI (HCAI)

1. **Feature Proposal:** Incorporate a Streamlit `st.expander` (dropdown) below each AI response, labeled 'View Textbook Source,' to enable *users to verify the source of the information*.
- **Operational Details:** When selected, the expander displays the specific paragraphs from ChromaDB that the AI uses. This feature demonstrates to students that the AI's responses are grounded in their course material rather than generated arbitrarily.

2. **Socratic Mode (Pedagogy):** A human tutor typically guides students toward discovering solutions independently, rather than providing direct answers. Find it yourself.

- **Feature Proposal: Integrate** a toggle switch in the Streamlit sidebar labeled 'Enable Socratic Mode,' allowing the tutor to provide hints rather than complete `answers`.
- **Operational Details:** When enabled, the Groq system prompt is modified to withhold full code solutions. Instead, the AI provides guided hints, such as: *"I see the issue with your list syntax. Look closely at line 3—what do you think is missing before the bracket?"* This approach encourages independent problem-solving and reduces over-reliance on the AI.

3. Emotional Reflection Loop (User Agency)

While the application infers user emotion from text, inaccuracies may occur, or explanations may not alleviate user anxiety. HCAI emphasizes user agency. Feature Proposal: After complex explanations, use Streamlit columns (`st.columns`) to place two buttons below the chat: [ I feel better about this] and [ I'm still confused/frustrated], allowing users to communicate their emotional state.

- **Operational Details:** If the user selects the sad face, the AI adapts its strategy by offering an apology, reducing technical jargon, and providing explanations through analogies (for example, comparing a Python list to a row of mailboxes) rather than code.

4. The "Learning Dashboard" (Encouraging Progress)

Students experiencing anxiety frequently focus on perceived knowledge gaps. A human-centered application should emphasize its progress. Feature Proposal: Add a sidebar section titled ' Today's Wins.'

- **Operational Details:** Maintain a hidden list in `st.session_state` tracking topics the student has successfully inquired about. For example, if the student asks about loops, lists, and functions, display a checklist in the user interface to illustrate their learning achievements during the session. Visualizing progress can help mitigate academic. Implementing even one or two of these features can transform the project from a standard RAG script into a thoughtful, psychologically aware educational tool, providing students with a clearer and more supportive learning experience. way to learn.